

## DSA -- Linked List, Stack & Queue

Target: Name all 4 linked list types in 10s. Explain LIFO vs FIFO with use cases in 20s.  
Know Infix/Prefix/Postfix cold -- it's a classic output question.

### Linked List

A linked list is a linear data structure where elements are NOT stored at contiguous memory locations.

Dynamic data structure -- size grows/shrinks at runtime

Each element = a node with two parts:

info stores the data

pointer stores address of the next node

Last node points to NULL

Start [10| ] [20| ] [30|NULL]  
Node Node Node

Node structure in C:

C

```
struct node {
```

```
    int data;
```

```
    struct node* next;
```

```
};
```

Array vs Linked List:

### Feature

Array

Linked List

### Memory

Contiguous

Non-contiguous

### Size

Fixed

Dynamic

### Access

O(1) random

O(n) sequential

### Insert/Delete at head

O(n) -- shift needed

O(1)

### Memory waste

Possible (fixed size)

No waste

### 4 Types of Linked List

#### 1. Singly Linked List

Each node has data + one pointer (to next node)

Unidirectional -- can only traverse forward

Last node NULL

Start [10| ] [20| ] [30|NULL]

#### 2. Doubly Linked List

Each node has data + two pointers (prev + next)

Bidirectional -- traverse forward AND backward

Last [10|prev|next] [20|prev|next] [30|prev|next] NULL

### 3. Circular Linked List

Last node's pointer points back to first node (not NULL)  
No clear start or end

Start [10| ] [20| ] [30| ]

### 4. Circular Doubly Linked List

Combination of Doubly + Circular  
Last node next first node; First node prev last node

### 4 Operations on Linked List

Operation  
What it does

Creation  
Create a node and link it to another node

Insertion  
Add a new node at beginning / end / specific position

Deletion  
Remove a node from the list

Traversal  
Visit every node from one end to the other

Applications of Linked List:  
- Dynamic memory allocation  
- Implementing Stacks and Queues  
- Undo functionality in software  
- Hash tables, Graphs

Stack  
A stack is a non-primitive, linear data structure that follows LIFO -- Last In, First Out.

The last element added is the first element removed. Think of a stack of plates.

PUSH [ 4 ] TOP  
      [ 3 ]  
      [ 2 ]  
      [ 1 ]  
      [ 0 ]

Two Operations

Operation  
Action  
Effect on TOP

PUSH  
Add element to top  
 $TOP = TOP + 1$

POP  
Remove element from top  
 $TOP = TOP - 1$

Stack Full (Overflow): PUSH when array is full no new element added  
Stack Underflow: POP when stack is empty TOP becomes negative

Applications of Stack:

- Function call management (call stack)
- Undo/Redo operations
- Expression evaluation (Infix Postfix)
- Backtracking algorithms

Stack Notations -- Infix, Prefix, Postfix

These are three ways to write mathematical expressions:

Notation

Operator position

Example

Also called

Infix

Between operands

$A + B$

Normal / Human notation

Prefix

Before operands

$+AB$

Polish Notation

Postfix

After operands

$AB+$

Reverse Polish / Suffix Notation

Mnemonic (Hinglish): "Infix mein operator In beech mein hai, Prefix mein Pehle hai, Postfix mein Peeche hai."

Operator Precedence (High Low)

$^$  (exponent) highest

$*$  / medium

$+$  - lowest

Left-to-right associativity for  $*$ ,  $/$ ,  $+$ ,  $-$ . Right-to-left for  $^$ .

Infix to Postfix -- Step-by-Step Algorithm

Scan expression left to right

Operand (A, B, number) write to output immediately

Operator push to stack, but first pop all operators of higher or equal precedence from stack to output

( always push to stack

) pop from stack to output until ( is found (discard both brackets)

End of expression pop all remaining operators from stack to output

Worked Example 1 -- Simple:  $A * B + C / D$

Step

Symbol

Action

Stack

Output

1  
A  
Operand   output  
[]  
A

2  
\*  
Push \*  
[\*]  
A

3  
B  
Operand   output  
[\*]  
A B

4  
+  
+ has lower precedence than \*   pop \* first, then push +  
[+]  
A B \*

5  
C  
Operand   output  
[+]  
A B \* C

6  
/  
/ has higher precedence than +   push  
[+ /]  
A B \* C

7  
D  
Operand   output  
[+ /]  
A B \* C D

8  
END  
Pop all: / then +  
[]  
A B \* C D / +

Result: A B \* C D / +

Worked Example 2 -- With brackets: (A + B) \* C

Step  
Symbol  
Action  
Stack  
Output

```

1
(
Push
[]

2
A
Output
[]
A

3
+
Push (inside bracket)
[( +]
A

4
B
Output
[( +]
A B

5
)
Pop until ( pop +
[]
A B +

6
*
Push
[*]
A B +

7
C
Output
[*]
A B + C

8
END
Pop *
[]
A B + C *

```

Result:  $A B + C *$  This is the ebook formula:  $(A+B)*C$

Infix to Prefix -- Algorithm

Reverse the expression convert to postfix reverse the result.

$(A + B) * C$

Step 1: Reverse  $C *) B + A ($

Step 2: Treat  $)$  as  $($  and vice versa convert to postfix  $C B A + *$

Step 3: Reverse result  $* + A B C$

Prefix result:  $* + A B C$

Quick Conversion Cheat

Infix  $A + B$  Postfix:  $AB+$  Prefix:  $+AB$

Infix  $A + B * C$  Postfix:  $ABC*+$  Prefix:  $+A*BC$

Infix  $(A+B)*C$  Postfix:  $AB+C*$  Prefix:  $*+ABC$

PSU Interview Answer (20 seconds)

"Infix is the normal notation -- operator between operands, like  $A+B$ . Prefix puts the operator before operands -- called

Polish notation, like +AB. Postfix puts operator after -- called Reverse Polish, like AB+. Computers prefer postfix because it's evaluated using a stack without needing precedence rules."

## Queue

A queue is a non-primitive, linear data structure that follows FIFO -- First In, First Out.

The first element added is the first element removed. Think of a line at a bank.

FRONT [10][20][30][40] REAR  
(dequeue) (enqueue)  
Initial state: F = -1, R = -1 (empty queue)  
- Each insert:  $R = R + 1$   
- Each delete:  $F = F + 1$   
5 Basic Queue Operations

## Operation

What it does

## Enqueue

Add element to the rear

## Dequeue

Remove element from the front

## Is Empty

Check if queue is empty

## Is Full

Check if queue is full

## Peek

Get front element without removing it

## Applications of Queue:

- CPU scheduling, Disk scheduling
- Handling interrupts in real-time systems
- BFS (Breadth First Search) graph traversal
- Call center / ticket systems

## Priority Queue

An extension of queue where each element has a priority.

Element with higher priority is dequeued before lower priority

If two elements have equal priority FIFO order applies

## Operation

Time Complexity

Insert (item, priority)

$O(\log n)$

Get highest priority

$O(1)$

Delete highest priority

$O(\log n)$

Heap is the preferred implementation for Priority Queue --  $O(\log n)$  insert/delete vs  $O(n)$  for array/list.

Circular Queue

A circular queue is one where the rear wraps around to the front when the end of the array is reached.

Solves the problem of wasted space in linear queues

Insertion of new element done at position 0 when queue is full at the end

Q[0]  
Q[3]    Q[1]   F = R = -1 (initially empty)  
Q[2]

Stack vs Queue -- The Classic Comparison

Feature

Stack

Queue

Order

LIFO (Last In First Out)

FIFO (First In First Out)

Add element

PUSH at top

Enqueue at rear

Remove element

POP from top

Dequeue from front

Real example

Stack of plates

Line at bank / ticket counter

Used in

Recursion, undo, expression eval

BFS, scheduling, buffers

Your 40-Second Script -- Stack vs Queue

"Stack follows LIFO -- last in, first out. You add and remove from the same end called TOP. Classic use cases: function call stack, undo operations, and expression evaluation. Queue follows FIFO -- first in, first out. You add at the rear and remove from the front. Classic use cases: CPU scheduling, BFS traversal, and buffering. The key difference is Stack has one end, Queue has two ends."

Follow-Up Questions

Q: What is stack overflow?

When you try to PUSH onto a full stack -- no more space to add. Common in recursive programs with no base case (infinite recursion).

Q: Why is Heap preferred over Array for Priority Queue?

Array needs  $O(n)$  to find the highest priority element. Heap maintains the max/min at the root always, so insert and delete are  $O(\log n)$ .

Q: What is the difference between Circular Queue and normal Queue?

In a normal array-based queue, once the rear reaches the end, the space before the front is wasted even if elements were dequeued. Circular queue wraps rear back to index 0, reusing that space.